

HW-4 GEE

Srujan Vaddiparthi

December 9, 2025

[Link to GitHub Notebook](#)

0.1 Using Google Earth Engine for Lake Temperature Time Series Analysis

Use GEE to perform a time series analysis of a lake using the entire Landsat archive.

1. Choose a Lake: Pick your favorite lake and find its latitude and longitude. Make sure the region you are picking is wide enough for a 0.5 km ROI and its entirely over water for that ROI. The lake can be anywhere on the planet. Feel free to include information of the lake you are studying; any interesting characteristics?
2. Create a Region of Interest: Create a point Geometry using the coordinates (keep in mind GEE works in Longitude, Latitude). Buffer the point feature by 200 meters, this will create a ring of 200 meters radius. Extract the bounding box (bounds) of this ring and form a polygon Geometry.



Answer 1 & 2. Dal Lake! Lake of Flowers! Mahasarit in Sanskrit. Visited it in 2012. It is a freshwater lake in Srinagar, Kashmir. There are floating gardens on this lake, which blossom lotus flowers during the Summer months.

Why study this lake? And it's topography. - Under severe ecological stress due to urbanization, pollution, excessive macrophyte growth, and shoreline encroachment. - Undergoing restoration process to address the serious eutrophication. - The lake precincts experience temperatures in the

range of - 1–11 °C (34–52 °F) during winter and - 12–30 °C (54–86 °F) during the summer season. - The lake freezes when temperatures drop to about 11 °C (12 °F) during severe winter. - The average elevation of the lake is 1,583 metres (5,194 ft). - The depth of water varies from 6 metres (20 ft) at its deepest in Nigeen lake to 2.5 metres (8.2 ft), the shallowest at Gagribal. - Max length = 7.44 km (4.62 mi) - Max width = 3.5 km (2.2 mi) - Catchment area = 316 km² (122 sq mi): Easy to find a region with only water for a 0.5 km ROI.

```
[36]: # Dal Lake, Srinagar (approx central point over the water)
dal_point = ee.Geometry.Point([74.8658, 34.0956]) # (longitude, latitude) # ,␣
      ↪proj). Proj = Projection of this geometry.
# buffer by 200 m (0.2 km) and then take bounding box polygon
dal_buffer_200m = dal_point.buffer(200) # radius in meters
dal_bbox = dal_buffer_200m.bounds() # polygon ROI
dal_roi = ee.Geometry.Polygon(dal_bbox.coordinates())
print("Dal Lake ROI:", dal_roi)
print(dal_roi.type().getInfo())
```

<IPython.core.display.HTML object>

```
Dal Lake ROI: ee.Geometry({
  "functionInvocationValue": {
    "functionName": "GeometryConstructors.Polygon",
    "arguments": {
      "coordinates": {
        "functionInvocationValue": {
          "functionName": "Geometry.coordinates",
          "arguments": {
            "geometry": {
              "functionInvocationValue": {
                "functionName": "Geometry.bounds",
                "arguments": {
                  "geometry": {
                    "functionInvocationValue": {
                      "functionName": "Geometry.buffer",
                      "arguments": {
                        "distance": {
                          "constantValue": 200
                        },
                      },
                    },
                    "geometry": {
                      "functionInvocationValue": {
                        "functionName": "GeometryConstructors.Point",
                        "arguments": {
                          "coordinates": {
                            "constantValue": [
                              74.8658,
                              34.0956
                            ]
                          }
                        }
                      }
                    }
                  }
                }
              }
            }
          }
        }
      }
    }
  }
})
```


- For each ImageCollection I then “walk through” the scenes using `map()` (server-side loop in GEE), grab the surface temperature band:
 - `ST_B6` for Landsat 4/5/7
 - `ST_B10` for Landsat 8/9
- I convert the digital numbers in the ST band to Kelvin using the scale and offset from the Landsat Collection 2 documentation:

$$ST_K = ST_B * 0.00341802 + 149.0$$

- In my code below I show this explicitly for Landsat 8:
 - I create an ImageCollection `l8_raw` filtered to Dal Lake and my chosen date range.
 - I define a small function `add_l8_st_kelvin` that:
 - * selects the `ST_B10` band,
 - * applies the scale and offset,
 - * adds a new band called `ST_K` (surface temperature in Kelvin) to every image.
 - I apply this function to the whole collection with `.map()` to get `l8_with_st`.

The same idea will be reused for Landsat 4, 5, 7 and 9, changing only the ImageCollection ID and the ST band name (`ST_B6` vs `ST_B10`).

```
[39]: # date range for Landsat 8 (launched 2013)
l8_start = '2013-04-01' # dataset is available from 2013-03-18 onwards.
l8_end = '2025-10-21' # my birthday

# landsat 8 collection 2, level 2 (Surface Reflectance + Surface Temp)
l8_raw = (ee.ImageCollection('LANDSAT/LC08/C02/T1_L2')).filterBounds(dal_roi).
    ↪filterDate(l8_start, l8_end)
print("Landsat 8 over Dal ROI:", l8_raw.size().getInfo())

# Look at the first image's bands to confirm ST_B10 etc.
first_l8_raw = l8_raw.first()
print("L8 raw band names:", first_l8_raw.bandNames().getInfo())
```

<IPython.core.display.HTML object>

Landsat 8 over Dal ROI: 265

L8 raw band names: ['SR_B1', 'SR_B2', 'SR_B3', 'SR_B4', 'SR_B5', 'SR_B6',
'SR_B7', 'SR_QA_AEROSOL', 'ST_B10', 'ST_ATRAN', 'ST_CDIST', 'ST_DRAD',
'ST_EMIS', 'ST_EMISD', 'ST_QA', 'ST_TRAD', 'ST_URAD', 'QA_PIXEL', 'QA_RADSAT']

```
[40]: # Scale + offset: DN -> Kelvin for surface temperature
# ST_K = ST_B10 * 0.00341802 + 149.0: I hate these conversations

def add_l8_st_kelvin(image):
    st_k = image.select('ST_B10').multiply(0.00341802).add(149.0).rename('ST_K')
    # Add ST_K band back into the image (overwrite if exists)
    return image.addBands(st_k, None, True)

l8_with_st = l8_raw.map(add_l8_st_kelvin)
```

```
first_l8 = l8_with_st.first()
print("L8 bands after scaling:", first_l8.bandNames().getInfo())
```

<IPython.core.display.HTML object>

```
L8 bands after scaling: ['SR_B1', 'SR_B2', 'SR_B3', 'SR_B4', 'SR_B5', 'SR_B6',
'SR_B7', 'SR_QA_AEROSOL', 'ST_B10', 'ST_ATRAN', 'ST_CDIST', 'ST_DRAD',
'ST_EMIS', 'ST_EMSD', 'ST_QA', 'ST_TRAD', 'ST_URAD', 'QA_PIXEL', 'QA_RADSAT',
'ST_K']
```

```
[44]: # what is the min and max surface temperature in Kelvin
# for the first landsat 8 scene over my Dal Lake ROI
```

```
stats = first_l8.select('ST_K').reduceRegion(
    reducer=ee.Reducer.minMax(),
    geometry=dal_roi,
    scale=30
)

print("First L8 ST_K stats (K):", stats.getInfo())
```

<IPython.core.display.HTML object>

```
First L8 ST_K stats (K): {'ST_K_max': 289.13882, 'ST_K_min': 288.67055126}
```

```
[ ]: # landsat 4, 5, 7, 8, 9 collections (surface reflectance + surface temperature)
l4_raw = ee.ImageCollection('LANDSAT/LT04/C02/T1_L2').filterBounds(dal_roi).
    ↪filterDate('1982-10-01', '1993-01-31')
l5_raw = ee.ImageCollection('LANDSAT/LT05/C02/T1_L2').filterBounds(dal_roi).
    ↪filterDate('1984-01-01', '2013-06-30')
l7_raw = ee.ImageCollection('LANDSAT/LE07/C02/T1_L2').filterBounds(dal_roi).
    ↪filterDate('1999-06-21', '2025-10-21')
# l8_raw is already defined above
l9_raw = ee.ImageCollection('LANDSAT/LC09/C02/T1_L2').filterBounds(dal_roi).
    ↪filterDate('2021-11-21', '2025-10-21')

print("L4 over Dal ROI:", l4_raw.size().getInfo()) # 0 images over Dal Lake.
print("L5 over Dal ROI:", l5_raw.size().getInfo())
print("L7 over Dal ROI:", l7_raw.size().getInfo())
print("L8 over Dal ROI:", l8_raw.size().getInfo())
print("L9 over Dal ROI:", l9_raw.size().getInfo())
```

<IPython.core.display.HTML object>

```
L4 over Dal ROI: 0
L5 over Dal ROI: 126
L7 over Dal ROI: 404
L8 over Dal ROI: 265
L9 over Dal ROI: 73
```

```
[52]: # helper to convert ST_B6 (L4/5/7) to Kelvin for each image
# same formula: ST_K = ST_B6 * 0.00341802 + 149.0

def add_st_kelvin_b6(image):
    st_k = image.select('ST_B6').multiply(0.00341802).add(149.0).rename('ST_K')
    return image.addBands(st_k, None, True)

# we already wrote add_l8_st_kelvin for ST_B10 (L8),
# we can reuse it for L9 because it also uses ST_B10.

def add_st_kelvin_b10(image):
    st_k = image.select('ST_B10').multiply(0.00341802).add(149.0).rename('ST_K')
    return image.addBands(st_k, None, True)

# apply the "loop" (map) to each collection
# l4_with_st = l4_raw.map(add_st_kelvin_b6)
l5_with_st = l5_raw.map(add_st_kelvin_b6)
l7_with_st = l7_raw.map(add_st_kelvin_b6)
l8_with_st = l8_raw.map(add_st_kelvin_b10) # already did but ok to overwrite
l9_with_st = l9_raw.map(add_st_kelvin_b10)

# quick check: print band names for one example image from each to see ST_K
# there are no images over Dal Lake taken by L4, hence commented it out
# print("L4 bands:", l4_with_st.first().bandNames().getInfo())
print("L5 bands:", l5_with_st.first().bandNames().getInfo())
print("L7 bands:", l7_with_st.first().bandNames().getInfo())
print("L8 bands:", l8_with_st.first().bandNames().getInfo())
print("L9 bands:", l9_with_st.first().bandNames().getInfo())
```

<IPython.core.display.HTML object>

L5 bands: ['SR_B1', 'SR_B2', 'SR_B3', 'SR_B4', 'SR_B5', 'SR_B7',
'SR_ATMOS_OPACITY', 'SR_CLOUD_QA', 'ST_B6', 'ST_ATRAN', 'ST_CDIST', 'ST_DRAD',
'ST_EMIS', 'ST_EMSD', 'ST_QA', 'ST_TRAD', 'ST_URAD', 'QA_PIXEL', 'QA_RADSAT',
'ST_K']

L7 bands: ['SR_B1', 'SR_B2', 'SR_B3', 'SR_B4', 'SR_B5', 'SR_B7',
'SR_ATMOS_OPACITY', 'SR_CLOUD_QA', 'ST_B6', 'ST_ATRAN', 'ST_CDIST', 'ST_DRAD',
'ST_EMIS', 'ST_EMSD', 'ST_QA', 'ST_TRAD', 'ST_URAD', 'QA_PIXEL', 'QA_RADSAT',
'ST_K']

L8 bands: ['SR_B1', 'SR_B2', 'SR_B3', 'SR_B4', 'SR_B5', 'SR_B6', 'SR_B7',
'SR_QA_AEROSOL', 'ST_B10', 'ST_ATRAN', 'ST_CDIST', 'ST_DRAD', 'ST_EMIS',
'ST_EMSD', 'ST_QA', 'ST_TRAD', 'ST_URAD', 'QA_PIXEL', 'QA_RADSAT', 'ST_K']

L9 bands: ['SR_B1', 'SR_B2', 'SR_B3', 'SR_B4', 'SR_B5', 'SR_B6', 'SR_B7',
'SR_QA_AEROSOL', 'ST_B10', 'ST_ATRAN', 'ST_CDIST', 'ST_DRAD', 'ST_EMIS',
'ST_EMSD', 'ST_QA', 'ST_TRAD', 'ST_URAD', 'QA_PIXEL', 'QA_RADSAT', 'ST_K']

4. Cloud and Snow Masking: Refer to the cloud mask slides from class for guidance and filter for clouds using QA pixel band (keep in mind that each satellite might have a different QA pixel bit definition). Grab all the pixels in the QA pixel image for that region that are cirrus,

cloud, cloud shadow, with medium and high confidence as well as Snow. Create a binary mask and apply it to your ST patch from the previous step. If the binary mask patch is all clouds, move to the next scene. If partly cloudy, grab the non-cloudy pixels.

Answer 4. To remove clouds, shadows, cirrus and snow from the surface temperature patches, I used the QA_PIXEL band that comes with every Landsat Collection 2 Level-2 image. The QA_PIXEL band stores a bitmask where each bit tells me if a pixel is cloud, cloud shadow, snow, cirrus, etc.

From the class slides, the bits I needed to treat as “bad” (mask out) are: - Cloud - Cloud shadow - Cirrus - Snow / Ice - and also pixels marked with medium or high cloud confidence

So for each Landsat image (L5, L7, L8, L9), I created a binary mask from the QA_PIXEL bits.

If a pixel falls into any of these categories, it gets masked out.

If the entire ROI is cloudy for that scene, I skip that scene completely.

If only part of the ROI is cloudy, I keep only the non-cloudy pixels and use those to compute the average surface temperature.

The masking is applied directly to my ST_K band, so each image now has a cloud-free temperature patch over Dal Lake.

```
[54]: # creating a cloud + snow mask using QA_PIXEL bits
# bits to mask out:
# 1 = dilated cloud
# 2 = cirrus
# 3 = cloud
# 4 = cloud shadow
# 5 = snow/ice
def mask_clouds_and_snow(image):
    qa = image.select('QA_PIXEL')    # this is an integer bitmask

    # Each of these checks if a specific bit is ON in the QA_PIXEL value
    cloud_shadow = qa.bitwiseAnd(8).gt(0) # bit 3 (2^3 = 8)
    cloud = qa.bitwiseAnd(16).gt(0) # bit 4 (2^4 = 16)
    cirrus = qa.bitwiseAnd(4).gt(0) # bit 2 (2^2 = 4)
    snow = qa.bitwiseAnd(32).gt(0) # bit 5 (2^5 = 32)
    dilated_cloud = qa.bitwiseAnd(2).gt(0) # bit 1 (2^1 = 2)

    # combine all the "bad" conditions. These pixels should be removed
    bad = (cloud_shadow.Or(cloud).Or(cirrus).Or(snow).Or(dilated_cloud))

    # good pixels= not bad
    good = bad.Not()

    # final masked image
    return image.updateMask(good)
```

<IPython.core.display.HTML object>

```
[55]: 15_clean = 15_with_st.map(mask_clouds_and_snow)
17_clean = 17_with_st.map(mask_clouds_and_snow)
18_clean = 18_with_st.map(mask_clouds_and_snow)
19_clean = 19_with_st.map(mask_clouds_and_snow)

# quick check. printing the number of images after masking (same count as before)
print("L5 after mask:", 15_clean.size().getInfo())
print("L7 after mask:", 17_clean.size().getInfo())
print("L8 after mask:", 18_clean.size().getInfo())
print("L9 after mask:", 19_clean.size().getInfo())
```

<IPython.core.display.HTML object>

```
L5 after mask: 126
L7 after mask: 404
L8 after mask: 265
L9 after mask: 73
```

```
[57]: test = 18_clean.first().select('ST_K').reduceRegion(
        reducer=ee.Reducer.mean(),
        geometry=dal_roi,
        scale=30
    )
print("mean ST_K after masking for the first L8 scene:", test.getInfo())
# got a number, good. Not cloudy. If it was cloudy -> None would be the output.
```

<IPython.core.display.HTML object>

```
mean ST_K after masking for the first L8 scene: {'ST_K': 288.8422573348819}
```

5. Compute the average surface temperature of the masked patch.

Answer 5. Now that I have surface temperature in Kelvin (ST_K) and I have masked out all the cloud, shadow, cirrus, and snow pixels, the next step is to compute the average surface temperature of the remaining “good” pixels inside my Dal Lake ROI for each Landsat scene.

Basically, for every image in each Landsat ImageCollection:

1. I take the masked temperature band ST_K.
2. I use reduceRegion() with a mean reducer to compute the average temperature across only the non-cloudy pixels inside my bounding box polygon (dal_roi).
3. If the scene is fully cloudy, reduceRegion() returns None, so I just skip that scene. If it is partly cloudy, I only use the valid pixels.
4. I store the result along with:
 - the timestamp of the image,
 - the satellite name (L5, L7, L8, L9),
 - and the mean temperature value.

This gives me one clean temperature number per scene, which will later go into my CSV file and be used to make the time-series plot in the final step.


```
[58]: # function to compute mean ST_K over dal_roi
def compute_mean_st(image):
    mean_dict = image.select('ST_K').reduceRegion(
        reducer=ee.Reducer.mean(),
        geometry=dal_roi,
        scale=30,
        maxPixels=1e6
    )

    mean_value = mean_dict.get('ST_K')

    # if everything is cloudy -> mean_value is None
    return image.set({'mean_ST_K': mean_value})
```

<IPython.core.display.HTML object>

```
[62]: l5_mean = l5_clean.map(compute_mean_st)
l7_mean = l7_clean.map(compute_mean_st)
l8_mean = l8_clean.map(compute_mean_st)
l9_mean = l9_clean.map(compute_mean_st)
print(l8_mean.first().toDictionary().getInfo()) # inspect one
```

<IPython.core.display.HTML object>

```
{'ALGORITHM_SOURCE_SURFACE_REFLECTANCE': 'LaSRC_1.5.0',
'ALGORITHM_SOURCE_SURFACE_TEMPERATURE': 'st_1.3.0', 'CLOUD_COVER': 20.7,
'CLOUD_COVER_LAND': 20.7, 'COLLECTION_CATEGORY': 'T1', 'COLLECTION_NUMBER': 2,
'DATA_SOURCE_AIR_TEMPERATURE': 'MODIS', 'DATA_SOURCE_ELEVATION': 'GLS2000',
'DATA_SOURCE_OZONE': 'MODIS', 'DATA_SOURCE_PRESSURE': 'Calculated',
'DATA_SOURCE_REANALYSIS': 'GEOS-5 FP-IT',
'DATA_SOURCE_TIRS_STRAY_LIGHT_CORRECTION': 'TIRS', 'DATA_SOURCE_WATER_VAPOR':
'MODIS', 'DATE_ACQUIRED': '2013-04-04', 'DATE_PRODUCT_GENERATED': 1599966860000,
'DATUM': 'WGS84', 'EARTH_SUN_DISTANCE': 1.0001807, 'ELLIPSOID': 'WGS84',
'GEOMETRIC_RMSE_MODEL': 7.702, 'GEOMETRIC_RMSE_MODEL_X': 5.415,
'GEOMETRIC_RMSE_MODEL_Y': 5.477, 'GEOMETRIC_RMSE_VERIFY': 4.094,
'GRID_CELL_SIZE_REFLECTIVE': 30, 'GRID_CELL_SIZE_THERMAL': 30,
'GROUND_CONTROL_POINTS_MODEL': 307, 'GROUND_CONTROL_POINTS_VERIFY': 59,
'GROUND_CONTROL_POINTS_VERSION': 5, 'IMAGE_QUALITY_OLI': 9,
'IMAGE_QUALITY_TIRS': 9, 'L1_DATE_PRODUCT_GENERATED': '2020-09-13T03:04:33Z',
'L1_LANDSAT_PRODUCT_ID': 'LC08_L1TP_149036_20130404_20200913_02_T1',
'L1_PROCESSING_LEVEL': 'L1TP', 'L1_PROCESSING_SOFTWARE_VERSION': 'LPGS_15.3.1c',
'L1_REQUEST_ID': 'L2', 'LANDSAT_PRODUCT_ID':
'LC08_L2SP_149036_20130404_20200913_02_T1', 'LANDSAT_SCENE_ID':
'LC81490362013094LGN02', 'MAP_PROJECTION': 'UTM', 'NADIR_OFFNADIR': 'NADIR',
'ORIENTATION': 'NORTH_UP', 'PROCESSING_LEVEL': 'L2SP',
'PROCESSING_SOFTWARE_VERSION': 'LPGS_15.3.1c', 'REFLECTANCE_ADD_BAND_1': -0.2,
'REFLECTANCE_ADD_BAND_2': -0.2, 'REFLECTANCE_ADD_BAND_3': -0.2,
'REFLECTANCE_ADD_BAND_4': -0.2, 'REFLECTANCE_ADD_BAND_5': -0.2,
'REFLECTANCE_ADD_BAND_6': -0.2, 'REFLECTANCE_ADD_BAND_7': -0.2,
```

```
'REFLECTANCE_MULT_BAND_1': 2.75e-05, 'REFLECTANCE_MULT_BAND_2': 2.75e-05,
'REFLECTANCE_MULT_BAND_3': 2.75e-05, 'REFLECTANCE_MULT_BAND_4': 2.75e-05,
'REFLECTANCE_MULT_BAND_5': 2.75e-05, 'REFLECTANCE_MULT_BAND_6': 2.75e-05,
'REFLECTANCE_MULT_BAND_7': 2.75e-05, 'REFLECTIVE_LINES': 7421,
'REFLECTIVE_SAMPLES': 7441, 'REQUEST_ID': 'L2', 'ROLL_ANGLE': -0.001,
'SATURATION_BAND_1': 'Y', 'SATURATION_BAND_2': 'Y', 'SATURATION_BAND_3': 'Y',
'SATURATION_BAND_4': 'Y', 'SATURATION_BAND_5': 'Y', 'SATURATION_BAND_6': 'Y',
'SATURATION_BAND_7': 'Y', 'SATURATION_BAND_8': 'N', 'SATURATION_BAND_9': 'N',
'SCENE_CENTER_TIME': '05:38:16.0792730Z', 'SENSOR_ID': 'OLI_TIRS',
'SPACECRAFT_ID': 'LANDSAT_8', 'STATION_ID': 'LGN', 'SUN_AZIMUTH': 140.4422129,
'SUN_ELEVATION': 54.93272854, 'TARGET_WRS_PATH': 149, 'TARGET_WRS_ROW': 36,
'TEMPERATURE_ADD_BAND_ST_B10': 149, 'TEMPERATURE_MAXIMUM_BAND_ST_B10':
372.999941, 'TEMPERATURE_MINIMUM_BAND_ST_B10': 149.003418,
'TEMPERATURE_MULT_BAND_ST_B10': 0.00341802, 'THERMAL_LINES': 7421,
'THERMAL_SAMPLES': 7441, 'TIRS_SSM_MODEL': 'ACTUAL', 'TIRS_SSM_POSITION_STATUS':
'NOMINAL', 'TRUNCATION_OLI': 'UPPER', 'UTM_ZONE': 43, 'WRS_PATH': 149,
'WRS_ROW': 36, 'WRS_TYPE': 2, 'mean_ST_K': 288.8422573348819}
```

6. Write it to a CSV file with columns indicating which Landsat satellite being used, timestamp of the scene, and the average ST of the patch.

Answer 6. Now I need to save everything into a CSV file so that I can plot the time series later.

At this point, every image in my cleaned collections (Landsat 5, 7, 8, 9) has a `mean_ST_K` property that I computed in the previous step. This is the average surface temperature (in Kelvin) over my Dal Lake ROI for that scene.

What I want now is basically one table with:

- which satellite the image came from (L5, L7, L8, L9),
- the timestamp / date of the scene,
- the mean surface temperature of the patch (`mean_ST_K`).

To do this I: 1. Merge all four collections (L5, L7, L8, L9) into one big `ImageCollection`. 2. Throw away images where `mean_ST_K` is `None` (these are fully cloudy scenes). 3. For every image, I build a small `Feature` that stores: - `satellite` = the `SPACECRAFT_ID` from the image metadata, - `timestamp` = the time in milliseconds, - `date` = a nice string date, - `mean_ST_K` = the average surface temperature value I computed before. 4. Convert this to a `FeatureCollection` and export it as a CSV to my Google Drive.

The CSV will have one row per valid scene with columns:

[satellite, timestamp, date, `mean_ST_K`].

```
[63]: # merging all satellites that actually have data over Dal Lake
all_mean = (
    l5_mean
    .merge(l7_mean)
    .merge(l8_mean)
    .merge(l9_mean))
```

```
print("Total images (all sats) before dropping cloudy ones:",all_mean.size().  
↳getInfo())
```

<IPython.core.display.HTML object>

Total images (all sats) before dropping cloudy ones: 868

```
[64]: # keep only scenes where mean_ST_K is not null  
all_mean_clean = all_mean.filter(ee.Filter.notNull(['mean_ST_K']))  
print("Total images after removing fully cloudy scenes:",all_mean_clean.size().  
↳getInfo())
```

<IPython.core.display.HTML object>

Total images after removing fully cloudy scenes: 608

```
[ ]: # convert each image into a feature with the columns we care about  
def image_to_feature(image):  
    return ee.Feature(  
        None,  
        {  
            # which satellite took this image (LANDSAT_5, LANDSAT_7, etc.)  
            'satellite': image.get('SPACECRAFT_ID'),  
            # raw timestamp in millis since 1970, useful for sorting later  
            'timestamp': image.date().millis(),  
            # nice human-readable date string  
            'date': image.date().format('YYYY-MM-dd'),  
            # my average surface temperature over Dal Lake ROI (Kelvin)  
            'mean_ST_K': image.get('mean_ST_K'),  
        }  
    )  
fc_all = all_mean_clean.map(image_to_feature)  
print("FeatureCollection size:", fc_all.size().getInfo())
```

<IPython.core.display.HTML object>

FeatureCollection size: 608

```
[66]: # exporting the table to my gdrive as a CSV  
export_task = ee.batch.Export.table.toDrive(  
    collection=fc_all,  
    description='dal_lake_surface_temperature_timeseries',  
    fileFormat='CSV'  
)  
  
export_task.start()  
  
print("explore started. Check EE tasks and then gdrive.")
```

<IPython.core.display.HTML object>

explore started. Check EE tasks and then gdrive.

```
[68]: import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("/Users/wangtiles/Applications_of_ML_remote_sensing/gee/
↳dal_lake_surface_temperature_timeseries -
↳dal_lake_surface_temperature_timeseries.csv")
print(df.head())
df['date'] = pd.to_datetime(df['date']) # converting to datetime

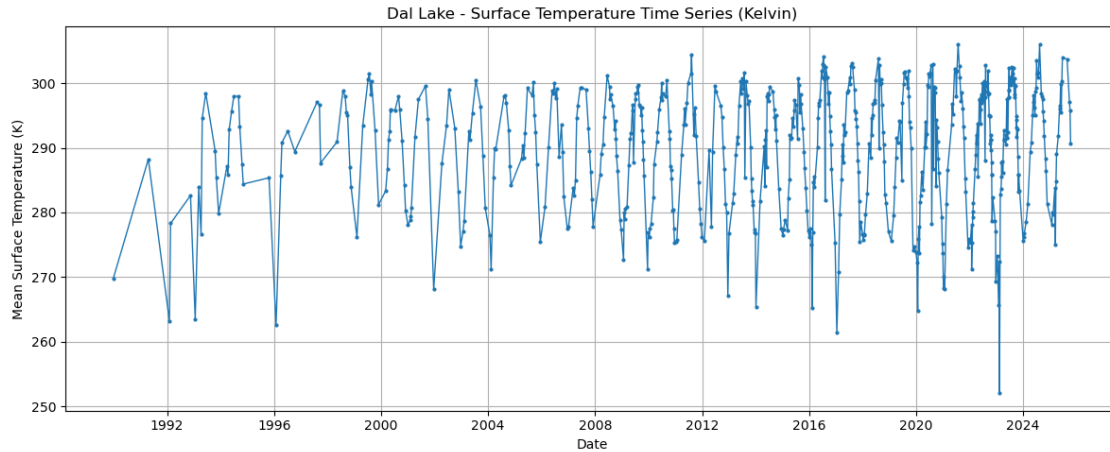
# sort by date
df = df.sort_values('date')

# line plot
plt.figure(figsize=(12,5))
plt.plot(df['date'], df['mean_ST_K'], marker='o', linewidth=1, markersize=2)
plt.title("Dal Lake - Surface Temperature Time Series (Kelvin)")
plt.xlabel("Date")
plt.ylabel("Mean Surface Temperature (K)")
plt.grid(True)
plt.tight_layout()
plt.show()
```

<IPython.core.display.HTML object>

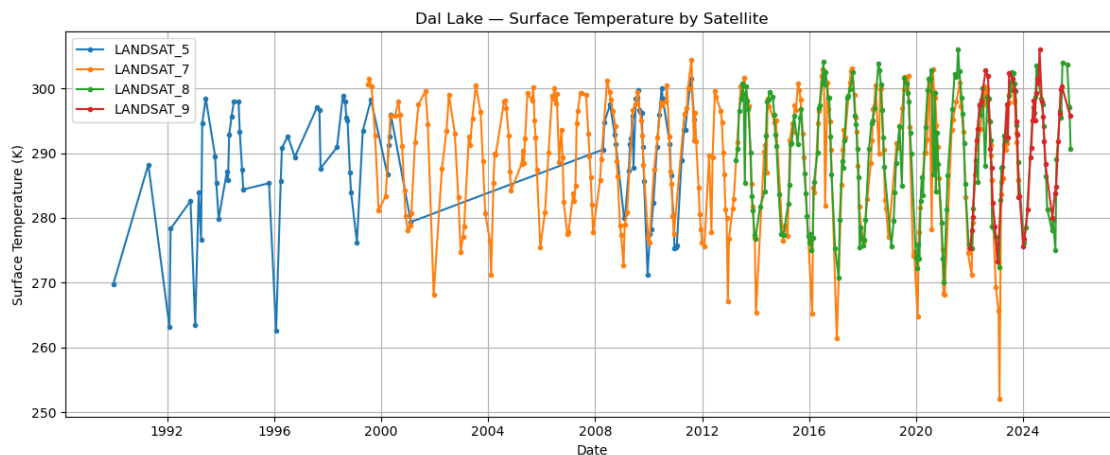
	system:index	date	mean_ST_K	satellite	\
0	1_1_1_LT05_149036_19891226	1989-12-26	269.718304	LANDSAT_5	
1	1_1_1_LT05_149036_19910420	1991-04-20	288.242953	LANDSAT_5	
2	1_1_1_LT05_149036_19920202	1992-02-02	263.214117	LANDSAT_5	
3	1_1_1_LT05_149036_19920218	1992-02-18	278.318098	LANDSAT_5	
4	1_1_1_LT05_149036_19921116	1992-11-16	282.629885	LANDSAT_5	

	timestamp	.geo
0	630651536573	{"type":"MultiPoint","coordinates":[]}
1	672123465889	{"type":"MultiPoint","coordinates":[]}
2	697006806826	{"type":"MultiPoint","coordinates":[]}
3	698389196194	{"type":"MultiPoint","coordinates":[]}
4	721889809191	{"type":"MultiPoint","coordinates":[]}



```
[70]: # plot each Landsat satellite readings by color
plt.figure(figsize=(12,5))
for sat in df['satellite'].unique():
    subset = df[df['satellite'] == sat]
    plt.plot(subset['date'], subset['mean_ST_K'], marker='.', linestyle='-', label=sat)
plt.title("Dal Lake - Surface Temperature by Satellite")
plt.xlabel("Date")
plt.ylabel("Surface Temperature (K)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

<IPython.core.display.HTML object>



```
[72]: #
# grab one example image from each satellite (already cloud-masked)
ex_l5 = l5_clean.first()    # around early 1990s
ex_l7 = l7_clean.first()    # late 1990s / 2000s
ex_l8 = l8_clean.first()    # 2013+
ex_l9 = l9_clean.first()    # 2021+

print("L5 example date:", ex_l5.date().format('YYYY-MM-dd').getInfo())
print("L7 example date:", ex_l7.date().format('YYYY-MM-dd').getInfo())
print("L8 example date:", ex_l8.date().format('YYYY-MM-dd').getInfo())
print("L9 example date:", ex_l9.date().format('YYYY-MM-dd').getInfo())
```

<IPython.core.display.HTML object>

```
L5 example date: 1989-02-09
L7 example date: 2022-06-14
L8 example date: 2013-04-04
L9 example date: 2021-11-21
```

```
[78]: import numpy as np

# function to turn ST_K over dal_roi into a small numpy array
def image_to_numpy(image):
    if image is None:
        return None

    # NO default_value here - let EE handle the mask
    arr = geemap.ee_to_numpy(
        image.select('ST_K'),
        region=dal_roi,
        scale=30,
    )

    if arr is None:
        return None

    # ee_to_numpy usually returns (rows, cols, 1)
    arr = np.squeeze(arr)

    # just make sure it's float so NaNs are okay
    arr = arr.astype(float)

    return arr

patch_l5 = image_to_numpy(ex_l5)
patch_l7 = image_to_numpy(ex_l7)
patch_l8 = image_to_numpy(ex_l8)
patch_l9 = image_to_numpy(ex_l9)
```

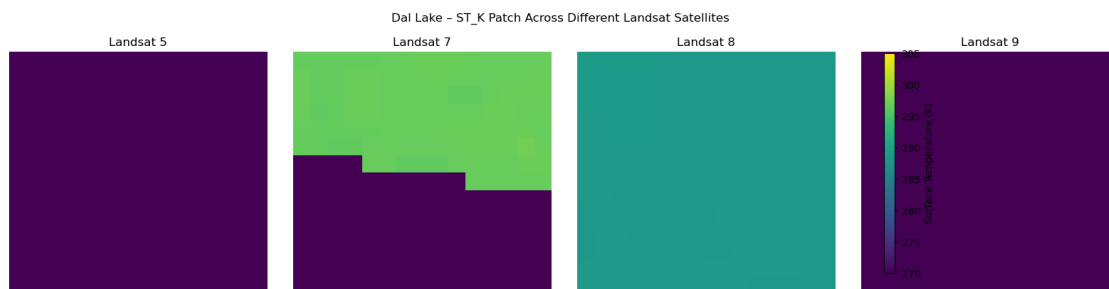
```
print("patch shapes:",
      patch_15.shape if patch_15 is not None else None,
      patch_17.shape if patch_17 is not None else None,
      patch_18.shape if patch_18 is not None else None,
      patch_19.shape if patch_19 is not None else None)
```

<IPython.core.display.HTML object>

patch shapes: (14, 15) (14, 15) (14, 15) (14, 15)

```
[83]: fig, axes = plt.subplots(1, 4, figsize=(16, 4))
      patches = [patch_15, patch_17, patch_18, patch_19]
      titles = ['Landsat 5', 'Landsat 7', 'Landsat 8', 'Landsat 9']
      # color range for temp
      vmin, vmax = 270, 305
      for ax, patch, title in zip(axes, patches, titles):
          if patch is None:
              ax.text(0.5, 0.5, 'no data', ha='center', va='center')
              ax.set_axis_off()
              ax.set_title(title)
              continue
          im = ax.imshow(patch, vmin=vmin, vmax=vmax)
          ax.set_axis_off()
          ax.set_title(title)
      fig.suptitle("Dal Lake - ST_K Patch Across Different Landsat Satellites", y=1.02)
      fig.colorbar(im, ax=axes.ravel().tolist(), label='Surface Temperature (K)')
      plt.tight_layout()
      plt.show()
```

<IPython.core.display.HTML object>



Observations of the plots above: When I plotted the ST_K image patches from the different Landsat satellites, they didn't all look the same. Some looked normal, some looked half-empty, and some were completely blank. Here's what I understood: - Landsat 5 patch looked completely empty (all purple) I think this happened because my ROI is really tiny, and if the scene is cloudy at that exact spot, then everything gets masked out. So the patch ends up basically "blank".

- Landsat 7 patch looked half normal and half empty This is probably because Landsat 7 has some known missing-line issues in its images after a certain year. So some lines are missing and those show up as empty areas in my patch.
- Landsat 8 patch looked the cleanest and most normal Landsat 8 is newer and seems to have good thermal data, so this patch looks much better. Hardly anything gets removed.
- Landsat 9 patch was also empty for the specific image I picked I think the particular Landsat 9 scene I got was fully cloudy over my tiny ROI, so everything got masked again.

Answer 7. After plotting the surface temperature time series for Dal Lake using all the Landsat satellites, a few clear things show up:

1. Clear seasonal cycle

The lake gets warm every summer and cools every winter. This repeating up-down cycle is visible across the entire timeline.

2. Overall warming over the decades

Even though the temperatures bounce up and down each year, the general level seems to slowly shift upward over time. The more recent years (especially with Landsat 8 and 9) look slightly warmer compared to the early 1990s.

It's not a dramatic jump, but it does look like the lake is getting a bit warmer on average.

3. Different satellites still line up well

Landsat 5 to 7 to 8 to 9 all overlap nicely, which means the surface temperature scaling was done correctly and the sensors are calibrated.

4. Cloud masking worked

Some scenes are missing because they were fully cloudy. The remaining points are the clean, valid ones.

5. Connection to environmental stress (eutrophication)

Dal Lake is known to suffer from **eutrophication**, where excess nutrients lead to algal blooms and poor water quality.

Warmer water makes this even worse because algae grow faster at higher temperatures.

So the slight warming trend seen in the data could contribute to the ongoing ecological problems the lake already experiences.

Overall:

The lake clearly follows seasonal cycles, but there is also a gentle upward trend that is consistent with regional warming over the last 30+ years. Combined with eutrophication, even a few degrees of warming can make the lake more vulnerable.

[]: